

Transacciones y Concurrency

ACID · schedules · anomalías · bloqueos · deadlock · timestamps · multiversión

Bases de Datos

Ing. Informática · Lic. en Cs. de Datos

Ricardo Di Pasquale

AGENDA

1. ¿Qué es una transacción? Propiedades ACID
2. Schedules (planes de ejecución)
3. Anomalías de la ejecución entrelazada
4. Bloqueos: Strict 2PL y deadlock
5. Demo en FreeSQL: bloqueo y deadlock
6. Manejo del deadlock y timestamps
7. Técnicas de multiversión



UCA

Pontificia Universidad Católica Argentina

**FACULTAD DE INGENIERÍA
Y CIENCIAS AGRARIAS**

PUNTO DE PARTIDA

**¿Qué es una
transacción?**

Una transacción es una interacción atómica

Varios pasos que se aplican uno tras otro, pero que el sistema debe tratar como una **única operación indivisible**: el estado intermedio no debe ser jamás visible ni quedar persistido.

Todo o nada: o se realiza completamente, o no debe tener ningún efecto.

El DBMS lo garantiza con una bitácora (log): registra las acciones para poder deshacerlas o rehacerlas.

El caso típico: transferencia entre cuentas



1. Verificar saldo suficiente en A.
2. Restar \$5.000 a A ($A = \15.000).
3. Sumar \$5.000 a B ($B = \5.000).

Si el sistema cae entre el paso 2 y el 3: queda $A = \$15.000$ y $B = \$0 \rightarrow$ se “volatilizan” \$5.000. Por eso: todo o nada.

Propiedades ACID



Atomicidad

Todas las acciones se completan o ninguna: nunca un estado intermedio.



Consistencia

Cada transacción parte de un estado consistente y deja otro consistente.



Aislamiento

Cada transacción corre como si estuviera sola; sin “ruido” de otras.



Durabilidad

Confirmada la transacción, sus efectos persisten aun ante fallas.

El log sostiene la Atomicidad (deshacer lo incompleto) y la Durabilidad (rehacer lo confirmado no volcado).

CÓMO SE ENTRELAZAN

Schedules (planes de ejecución)

Schedules: serial vs. entrelazado

Schedule = lista ordenada de acciones (R, W, commit, rollback) de varias transacciones, respetando el orden interno de cada una.

- ▶ **Completo:** incluye un commit o rollback por cada transacción.
- ▶ **Serial:** las transacciones no se mezclan: una entera y después la otra.
- ▶ **Entrelazado:** acciones de distintas transacciones intercaladas → puede haber anomalías.

T1	T2
R (a)	
W (a)	
Commit	
	R (b)
	W (b)
	Commit

serial: T1 → T2



Pontificia Universidad Católica Argentina

FACULTAD DE INGENIERÍA
Y CIENCIAS AGRARIAS

QUÉ PUEDE SALIR MAL

Anomalías de la ejecución entrelazada

Lectura sucia (dirty read)

T2 lee un dato que T1 modificó pero todavía no confirmó. Si T1 luego hace rollback, T2 trabajó sobre datos que “nunca existieron”.

T1	T2
W (a)	
	R (a)
	W (a)
	Commit
Rollback	

T2 leyó el a modificado por T1 y confirmó; el rollback de T1 deja la base inconsistente.

Lectura irrepetible

T1 lee a; T2 lo modifica y confirma; si T1 vuelve a leer a, obtiene otro valor — algo imposible en una ejecución serial.

T1	T2
R (a)	
	R (a)
	W (a)
	Commit
R (a) ¿?	

Ambas leen a=5; T1 quiere +1 y T2 -1. Serial siempre da 5; entrelazado, el resultado depende del orden y se pierde una actualización.

Overwriting uncommitted data

Dos empleados deben mantener el mismo salario. T1 los pone en \$3000 y T2 en \$4000; entrelazadas, cada registro queda con un valor distinto.

T1	T2
$W(emp1) = 3000$	
	$W(emp2) = 4000$
$W(emp2) = 3000$	
	$W(emp1) = 4000$

Resultado: emp1=\$4000 y emp2=\$3000 → inconsistente. Serial dejaría a ambos en \$3000 o ambos en \$4000.

Schedules recuperables

Recuperable: una transacción confirma sólo si todas aquellas de las que leyó datos ya confirmaron. Si T2 leyó de T1 y comitea antes que T1, y T1 hace rollback → **irrecuperable**.

T1	T2
W (a)	
	R (a)
	W (a)
	Commit
Rollback	

T2 ya confirmó: no se puede deshacer. Schedule irrecoverable.

CÓMO EVITARLAS

Bloqueos: Strict 2PL

Strict 2-Phase Locking (S2PL)

- ▶ **Leer un objeto:** pedir antes un shared lock (S) sobre él.
- ▶ **Escribir un objeto:** pedir antes un exclusive lock (X) sobre él.
- ▶ **Al terminar (commit/rollback):** se liberan TODOS los locks de la transacción.

S y X son incompatibles sobre el mismo objeto. **Cumpliendo estas reglas el DBMS entrelaza transacciones de forma segura** — pero S2PL por sí solo no evita deadlocks.

S2PL · un entrelazado seguro

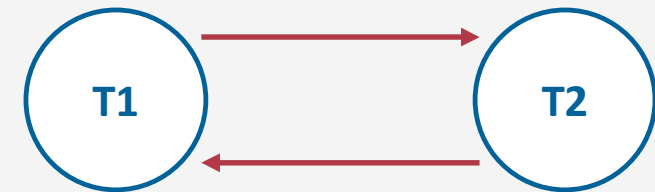
T1	T2
S (a)	
R (a)	
	S (a)
	R (a)
	X (b)
	R (b)
	W (b)
	Commit
X (c)	
R (c)	
W (c)	
Commit	

T1 y T2 comparten S(a) (lecturas compatibles); las escrituras (X) se serializan por objeto.

Deadlock: espera circular

Cada transacción tiene un lock que la otra necesita, y espera por el lock de la otra: ninguna avanza.

T1	T2
S (a)	S (b)
R (a)	R (b)
X (b) ...	X (a) ...
espera	espera



grafo de espera: ciclo $\Rightarrow$ deadlock

El S2PL conservador (pedir todos los candados al inicio) evita deadlocks, pero limita la concurrencia.

MANOS A LA OBRA

Demo en FreeSQL: bloqueo y deadlock

Preparar la tabla (una sola vez)

```
DROP TABLE IF EXISTS cuenta;  
CREATE TABLE cuenta (  
    id          INTEGER          PRIMARY KEY,  
    titular     VARCHAR2(30) NOT NULL,  
    saldo       NUMBER(12,2) NOT NULL );  
INSERT INTO cuenta VALUES (1, 'Armando', 20000);  
INSERT INTO cuenta VALUES (2, 'Benito', 0);  
COMMIT;
```

Ojo: la worksheet del navegador (Database Actions) es stateless y hace COMMIT implícito en cada Run. No sirve para ver locks. Usá dos sesiones SQLcl (o dos conexiones de SQL Developer desktop) contra tu instancia FreeSQL.

Bloqueo (lock wait) — dos sesiones

#	Sesión A	Sesión B
1	UPDATE cuenta SET saldo=saldo-5000 WHERE id=1; -- toma lock fila 1	
2		UPDATE cuenta SET saldo=saldo-1000 WHERE id=1; -- ESPERA a A
3	COMMIT; -- libera la fila 1	
4		(se desbloquea y aplica)

Los locks serializan: la Sesión B queda esperando hasta que A confirme o revierta.

Deadlock (ORA-00060) — dos sesiones

#	Sesión A	Sesión B
1	UPDATE ... saldo-5000 WHERE id=1;	
2		UPDATE ... saldo-3000 WHERE id=2;
3	UPDATE ... saldo+3000 WHERE id=2;	
	-- espera lock de B (fila 2)	
4		UPDATE ... saldo+5000 WHERE id=1;
		-- ciclo -> DEADLOCK

ORA-00060: deadlock detected while waiting for resource

Oracle revierte SOLO la sentencia víctima (no toda la transacción); la otra sesión avanza. Cada una debe COMMIT/ROLLBACK.



UCA

Pontificia Universidad Católica Argentina

FACULTAD DE INGENIERÍA
Y CIENCIAS AGRARIAS

PREVENIR, DETECTAR, RECUPERAR

Manejo del deadlock

Prevención de deadlocks

Técnica	Idea	Costo
S2PL conservador	pedir todos los candados antes de empezar	limita la concurrencia
Orden total de recursos	acceder siempre en el mismo orden	obliga a conocer el orden
esperar-morir (wait-die)	la vieja espera a la nueva; la nueva que choca, aborta	aborta y reinicia de más
herir-esperar (wound-wait)	la nueva espera a la vieja; la vieja desaloja a la nueva	aborta y reinicia de más

Con timestamps: en ambas reglas se aborta la más reciente. Libres de deadlock, pero con abortos innecesarios.

Detección y otras estrategias

- ▶ **No Waiting (NW):** si no obtiene el candado, aborta y reintenta tras un tiempo aleatorio.
- ▶ **Cautious Waiting (CW):** espera sólo si quien tiene el lock no está a su vez detenido → libre de deadlock.
- ▶ **Timeout:** aborta la transacción que espera un lock más que un umbral fijo.
- ▶ **Detección por grafo de espera:** arco $T_i \rightarrow T_j$ si T_i espera un recurso de T_j ; ciclo $\Rightarrow$ deadlock → se elige una víctima.

Riesgos: reinicio cíclico (siempre vuelve a deadlock), inanición (la víctima es siempre la misma) y espera indefinida. Se mitigan con colas FIFO / esquemas justos.

Control de concurrencia por Timestamp

Cada elemento X guarda **ReadTS(X)** y **WriteTS(X)** (el mayor timestamp que lo leyó / escribió). Se compara con $TS(T)$ para no violar el orden serial.

```
w(X):  si ReadTS(X) > TS(T) o WriteTS(X) > TS(T) -> abortar T (llega tarde)
        si no -> escribir y WriteTS(X) = TS(T)
r(X):  si WriteTS(X) > TS(T) -> abortar T (lee algo más nuevo)
        si no -> leer y ReadTS(X) = max(ReadTS(X), TS(T))
```

Garantiza seriabilidad y no tiene deadlocks, pero puede haber reinicio cíclico e inanición. **Regla de Thomas**: ante una escritura vieja ($WriteTS(X) > TS(T)$) la ignora en vez de abortar.

Técnicas de multiversión

Guardan versiones anteriores de cada dato. **Los lectores no bloquean a los escritores**: cada transacción lee la versión consistente que le corresponde por su timestamp.

Costo: más almacenamiento para mantener las versiones. Muchos motores comerciales combinan variantes — p. ej. los segmentos de rollback / MVCC de Oracle.

Es la base del aislamiento por lectura consistente que usan los RDBMS actuales.

Síntesis

- ▶ Transacción = unidad atómica; el DBMS garantiza ACID con el log.
- ▶ El entrelazado sin control produce anomalías (dirty/irrepetible/overwrite).
- ▶ Un schedule serial (o serializable) es el criterio de correctitud.
- ▶ S2PL da seguridad con locks S/X, pero habilita deadlocks.
- ▶ Deadlock: se previene, se detecta (grafo/timeout) o se evita (timestamps).
- ▶ Multiversión: lectores no bloquean escritores (MVCC).
- ▶ En Oracle el deadlock real se ve con dos sesiones (SQLcl), no en la web.

¡Muchas gracias! Q&A

Seguinos



@ingeagrariasuca



@IngeagrariasUCA



Ingeniería y Ciencias Agrarias

Contacto



Av. Alicia Moreau de Justo 1500
C1107AAZ, Buenos Aires, Argentina



0810-220-0822



inge_agrarias@uca.edu.ar